



Atenção

- Este relatório não é uma entrega separada: ele deve ser enviado junto com o .tar da atividade, no mesma atividade no Google Classroom.
- O relatório deve ter como nome as iniciais do seu e-mail em minúsculas (ex.: mla@cesar.school → diretório mla). Fora desse padrão, o relatório é descartado automaticamente.
- Cópia identificada, de outro(a) aluno(a) ou de qualquer outra fonte, descarta toda a submissão, código e relatório juntos.

1. Objetivo (3–5 linhas)

O projeto ProcessFlow tem como objetivo implementar, em C e ambiente Linux, um interpretador de comandos capaz de cadastrar e executar tarefas utilizando processos do sistema operacional. A solução desenvolvida permite execuções simples, sequenciais, paralelas e encadeadas por pipes, além de redirecionamento de entrada e saída, diretório de trabalho, execução em background e gerenciamento de jobs. O programa também pode operar de forma interativa ou executar comandos definidos em arquivos de workflow `.pf`.

2. Checklist de Requisitos

Um requisito por linha. Sem parágrafo, se precisar explicar algo, use a coluna "como testar".

Requisito do enunciado	Status	Como testar / evidência
Cadastrar tarefas com task	OK	task listar /bin/ls
Executar uma tarefa com run	OK	run listar
Execução sequencial	OK	run sequencial tarefa1 tarefa2
Execução paralela	OK	run parallel tarefa1 tarefa2
Pipeline entre tarefas	OK	run pipe tarefa1 tarefa2

Redirecionamento de entrada OK input tarefa arquivo.txt

Redirecionamento de saída	OK	output tarefa arquivo.txt
Saída em modo append	OK	append tarefa arquivo.txt
Definição do diretório de trabalho	OK	workdir /tmp seguido de uma task
Execução em background	OK	start tarefa
Listagem de jobs	OK	jobs
Espera de um job específico	OK	wait <jobId>
Modo interativo	OK	./processflow
Execução por workflow	OK	./processflow teste.pf
Uso de fork() e família exec()	OK	Execução das tasks cadastradas
Sincronização com wait()/waitpid()	OK	run, run sequencial, run parallel e wait
Pipes com pipe() e dup2()	OK	run pipe
Tratamento de erros	OK	Testes com task, job, arquivo, programa e workdir inválidos

3. Como Reproduzir

Comandos reais, copiáveis. Se não rodar com o que está aqui, não é avaliado.

Compilar:

```
make clean
make
```

Executar no modo interativo:

```
./processflow
```

Exemplo:

```
task listar /bin/ls
run listar
exit
```

Executar com workflow:

```
./processflow teste.pf
```

Exemplo:

```
task listar /bin/ls
task data /bin/date
run listar
run data
Exit

make test
```

4. Arquitetura (resumida)

Uma linha por arquivo, só o que é específico do seu projeto.

Arquivos e responsabilidades

1. `main.c` → interpreta os comandos, controla os modos interativo/workflow e direciona cada operação para as funções responsáveis.
2. `task.h` → define a estrutura `Task`, constantes e protótipos relacionados às tarefas.
3. `task.c` → implementa cadastro, busca e execução das tarefas, incluindo processos, redirecionamentos, pipes e diretório de trabalho.
4. `job.h` → define a estrutura `Job` e as operações disponíveis para gerenciamento de processos em background.
5. `job.c` → implementa criação, busca, atualização, listagem e espera dos jobs.
6. `Makefile` → automatiza compilação, execução, testes e limpeza dos arquivos gerados.

Decisões técnicas

1. Separar `Task` e `Job` em módulos → evitar concentrar toda a implementação em `main.c` → responsabilidades ficam mais organizadas e funções podem ser reutilizadas.
2. Utilizar `fork()` e `exec()` → executar programas externos de acordo com os requisitos da atividade → cada task é executada em um processo filho independente.
3. Utilizar `waitpid()` → controlar a sincronização entre pai e filhos → permite execuções bloqueantes, paralelas e gerenciamento de processos em background.

4. Utilizar `pipe()` e `dup2()` → conectar a saída de uma tarefa à entrada da próxima → possibilita pipelines sem recorrer a `system()` ou `popen()`.
5. Utilizar o mesmo interpretador para terminal e `.pf` → evitar duplicação da lógica de comandos → tanto a entrada interativa quanto workflows passam pelo mesmo processamento.

5. Estratégias e Diário de Desenvolvimento

Isto é o núcleo do relatório.

5.1 Estratégias da semana

Uma linha por abordagem diferente que você tentou. Se você não mudou de abordagem, preencha só S1.

Estratégia	Nome curto	Contexto	Motivo da troca (se houve)
S1	Construção incremental	Implementação inicial do interpretador, cadastro de tasks e execução simples com processos	As funcionalidades seguintes exigiam separar melhor execução, redirecionamento e controle dos processos
S2	Modularização e composição	Implementação de sequencial, parallel, redirecionamentos, pipes e workdir reutilizando as funções de Task	A execução em background passou a exigir armazenamento persistente de PID e estado
S3	Gerenciamento e robustez	Criação de Jobs, <code>start</code> , <code>jobs</code> , <code>wait</code> , workflows e tratamento dos casos de erro	Estratégia final utilizada para completar e validar os requisitos

5.2 Diário de Tentativas

Uma linha por tentativa relevante, marcando a qual estratégia (S1/S2/S3) ela pertence, obviamente precisa ser em ordem cronológica. A coluna "Quando" precisa bater com a ordem cronológica do `evidencias.log`, é isso que comprova que o fluxo é real. "Hipótese/Causa" só é preenchida quando o resultado foi falha ou parcial.

#	Estrat.	O que tentei	Resultado	Hipótese/Causa (se falhou)	Quando	Evidência
1	S1	Criar estrutura do interpretador e cadastro de tasks	Sucesso		Quarta	Compilação com <code>gcc -std=c11 -Wall -Wextra -pedantic</code> sem erros
2	S1	Implementar execução com <code>fork()</code> , <code>exec()</code> e <code>waitpid()</code>	Sucesso		Quinta	<code>task listar</code> , <code>task data</code> , <code>run data</code> , <code>run listar</code>

#	Estrat.	O que tentei	Resultado	Hipótese/Causa (se falhou)	Quando	Evidência
3	S1	Compilar cadastro utilizando <code>strdup()</code>	Falha	<code>strdup()</code> não estava disponível sob as opções de compilação utilizadas	Quinta	<code>run data e task eco /bin/echo</code> TESTE
4	S2	Implementar execução sequencial, paralela e redirecionamentos	Sucesso		Sexta	<code>run sequentia l data listar e run parallel dormir3 dormir1</code>
5	S2	Implementar pipeline entre tarefas	Parcial → sucesso	Argumento incorreto usado no teste do <code>wc</code> e ajustes posteriores na implementação	Domingo	<code>run pipe listar contar</code> retornou 9
6	S2	Implementar <code>workdir</code> integrado às diferentes formas de execução	Parcial → sucesso	Assinaturas das funções e propagação do diretório precisaram ser ajustadas	Domingo	<code>output listar, append data, input mostrar, com conteúdo confirmado</code>
7	S3	Implementar <code>start</code> , <code>jobs</code> e <code>wait</code>	Sucesso após ajustes	Funções de <code>jobs</code> precisaram ser declaradas e implementadas de forma consistente	Segunda	<code>start dormir3, jobs, wait 2, jobs</code>
8	S3	Executar comandos através do arquivo <code>.pf</code>	Parcial → sucesso	Arquivo com <code>\r\n</code> deixava <code>\r</code> nos tokens; leitura foi ajustada para remover <code>\r\n</code>	Segunda	<code>./process flow teste.pf executou task, run e exit</code>

Regra prática: se algo deu muito errado (bug que te travou por horas), a linha da tabela resume em 1 frase e aponta para o trecho do `evidencias.log` com o detalhe.

6. Evidências

A regra continua: toda afirmação relevante precisa de prova. O que muda é como capturar essa prova, para não depender de print manual toda hora.

6.1 Log automático (faz o trabalho pesado por você)

No início da sessão de trabalho, rode:

```
script -a evidencias.log
```

Isso grava automaticamente todos os comandos e saídas do terminal num arquivo, sem você precisar tirar print de cada linha. Comece a sessão com `date`, `whoami` e `pwd`, assim o log já tem timestamp, usuário e diretório sem esforço extra. **O evidencias.log deve ser enviado dentro do .tar**

6.2 Prints

```
luizv@Luizvieira:/mnt/c/Users/luizv/Desktop/3º periodo/processflow$ make
gcc -std=c11 -Wall -Wextra -pedantic src/main.c src/task.c -o processflow
src/task.c: In function 'cadastrar_task':
src/task.c:34:13: error: implicit declaration of function 'strdup'; did you mean 'strcmp'? [-Wimplicit-function-declaration]
 34 |         strdup(tokens[i]);
    |         ^~~~~~
    |         strcmp
src/task.c:33:55: error: assignment to 'char *' from 'int' makes pointer from integer without a cast [-Wint-conversion]
 33 |         task->argumentos[task->quantidade_argumentos] =
    |                                                         ^
make: *** [Makefile:10: processflow] Error 1
```

Durante a implementação do cadastro de tarefas, a compilação com `-std=c11 -Wall -Wextra -pedantic` apresentou erro por declaração implícita de `strdup()`. Como a função não faz parte do padrão C11 estrito utilizado na compilação, a abordagem foi substituída pela alocação manual de memória com `malloc()` e cópia da string com `strcpy()`. Após a alteração, a compilação prosseguiu normalmente.

```
processflow> run final
PROCESSFLOW_OK
processflow> exit
luizv@Luizvieira:/mnt/c/Users/luizv/Desktop/3º periodo/processflow$ date
Mon Aug 24 18:45:54 -03 2026
luizv@Luizvieira:/mnt/c/Users/luizv/Desktop/3º periodo/processflow$ exit
exit
Script done.
luizv@Luizvieira:/mnt/c/Users/luizv/Desktop/3º periodo/processflow$ ls -lh evidencias.log
-rwxrwxrwx 1 luizv luizv 7.8K Aug 24 18:45 evidencias.log
luizv@Luizvieira:/mnt/c/Users/luizv/Desktop/3º periodo/processflow$ make clean
rm -f processflow
luizv@Luizvieira:/mnt/c/Users/luizv/Desktop/3º periodo/processflow$ make
gcc -std=c11 -Wall -Wextra -pedantic src/main.c src/task.c src/job.c -o processflow
luizv@Luizvieira:/mnt/c/Users/luizv/Desktop/3º periodo/processflow$ ./processflow
processflow> task final /bin/echo PROCESSFLOW_OK
Task 'final' cadastrada.
processflow> run final
PROCESSFLOW_OK
processflow> exit
luizv@Luizvieira:/mnt/c/Users/luizv/Desktop/3º periodo/processflow$
```

Obrigatório preencher mesmo se a resposta for "não"

Onde usei IA (2–3 linhas):

Prompts principais (liste, sem história):

O que validei manualmente e como:

8. Reflexão Final (5–8 linhas)

O que você melhoraria com mais tempo, principais decisões, sem subseções separadas.

Reflexão:

Validação final (o print do meu computador bugou)

7. Uso de IA

Obrigatório preencher mesmo se a resposta for "não utilizei".

Onde usei IA:

Utilizei ferramentas de IA como apoio durante o desenvolvimento, principalmente para compreender conceitos de processos em Linux, identificar erros de compilação e auxiliar na implementação incremental das funcionalidades. Também utilizei IA na revisão final do código, no tratamento de casos de erro e na organização do relatório.

Prompts principais:

- “Ensine-me essa atividade passo a passo, começando pela estrutura do projeto e fazendo primeiro o task, run, fork() e exec().”
- “Agora que está no terminal Linux, muda algo no código?”
- “Vamos iniciar a etapa de execução paralela.”
- “Vamos fazer a etapa de pipes.”
- “Onde adicionar o waitpid() com WNOHANG?”
- “Faça uma lista de erros que eu preciso tratar.”
- “Revise o projeto inteiro com base na checklist de erros e requisitos.”

O que validei manualmente e como:

Validei manualmente o código compilando o projeto no WSL com make clean e make e executando as funcionalidades diretamente pelo terminal. Foram testados cadastro e execução de tasks, execução sequencial e paralela, pipes, redirecionamentos, diretório de trabalho, execução em background, jobs, wait e workflows .pf. Também testei entradas inválidas, como task e job inexistentes, programa não executável, diretório inválido, workflow inexistente e quantidade incorreta de argumentos. A sessão final de testes foi registrada no arquivo evidencias.log.

8. Reflexão Final (5–8 linhas)

O que você melhoraria com mais tempo, principais decisões técnicas e erros que evitaria na próxima. Tudo junto, sem subseções separadas.

Reflexão:

A implementação permitiu compreender de forma prática como processos são criados, executados e sincronizados em Linux. Uma das principais decisões foi separar o gerenciamento de tarefas e jobs em módulos, evitando concentrar toda a lógica no interpretador principal. A implementação de execução paralela e pipes ajudou a entender melhor a relação entre fork(), exec(), waitpid(), pipe() e dup2(). Durante o desenvolvimento, erros envolvendo strdup(), declarações de funções e diferenças entre os ambientes Windows e Linux mostraram a importância de compilar e testar frequentemente. Em uma próxima implementação, eu definiria a arquitetura dos módulos e suas interfaces antes de iniciar as funcionalidades mais complexas. Também criaria testes automatizados desde as primeiras etapas, em vez de concentrar a validação completa no final do projeto.

9. Checklist Final de Entrega

Item	Confirmado?
Compilei do zero seguindo só o que está no relatório	Sim
Rodei os testes e evidencias.log foi gerado	Sim
Tenho prints obrigatórios	Sim
Testei pelo menos um caso limite e um caso inválido	Sim

Item	Confirmado?
Preenchi a seção de uso de IA	Sim
Revisei o relatório e removi frases genéricas / vazias	Sim

10. Se eu tivesse mais 2 horas

Aplique as instruções abaixo de forma objetiva e verificável, evitando texto genérico.

Escreva 5-10 linhas sobre o que você melhoraria (robustez, testes, warnings, performance, organização).

Se eu tivesse mais duas horas para trabalhar, focaria principalmente na organização do projeto. Apesar de todas as funcionalidades estarem implementadas, algumas partes do código cresceram bastante conforme novos comandos foram adicionados. Eu reorganizaria principalmente a main.c, separando a interpretação dos comandos em funções menores para deixar o código mais simples de entender e manter. Também revisaria os nomes de funções e variáveis para manter um padrão em todo o projeto e adicionaria comentários apenas nos pontos mais importantes. Por fim, organizaria melhor os arquivos de teste e workflows, deixando a estrutura do projeto mais limpa para que outra pessoa consiga entender rapidamente como cada parte funciona.

Regras de Integridade (não mudam)

- Ocorrência relevante sem evidência (log ou print) = não conta para a nota.
- Contradição entre prints, log e relato = relatório perde credibilidade.
- Indício de evidência fabricada = entrega invalidada.